

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

AIM : Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.

CODE CLI:

```
print("Diffie-Hellman Key Exchange")

p = int(input("Enter a prime number (p): "))
g = int(input("Enter a primitive root (g): "))

a = int(input("Enter Shubham's private key: "))
b = int(input("Enter Rohan's private key: "))

A = pow(g, a, p)
B = pow(g, b, p)

print("\nShubham's Public Key:", A)
print("Rohan's Public Key:", B)

shubham_key = pow(B, a, p)
rohan_key = pow(A, b, p)

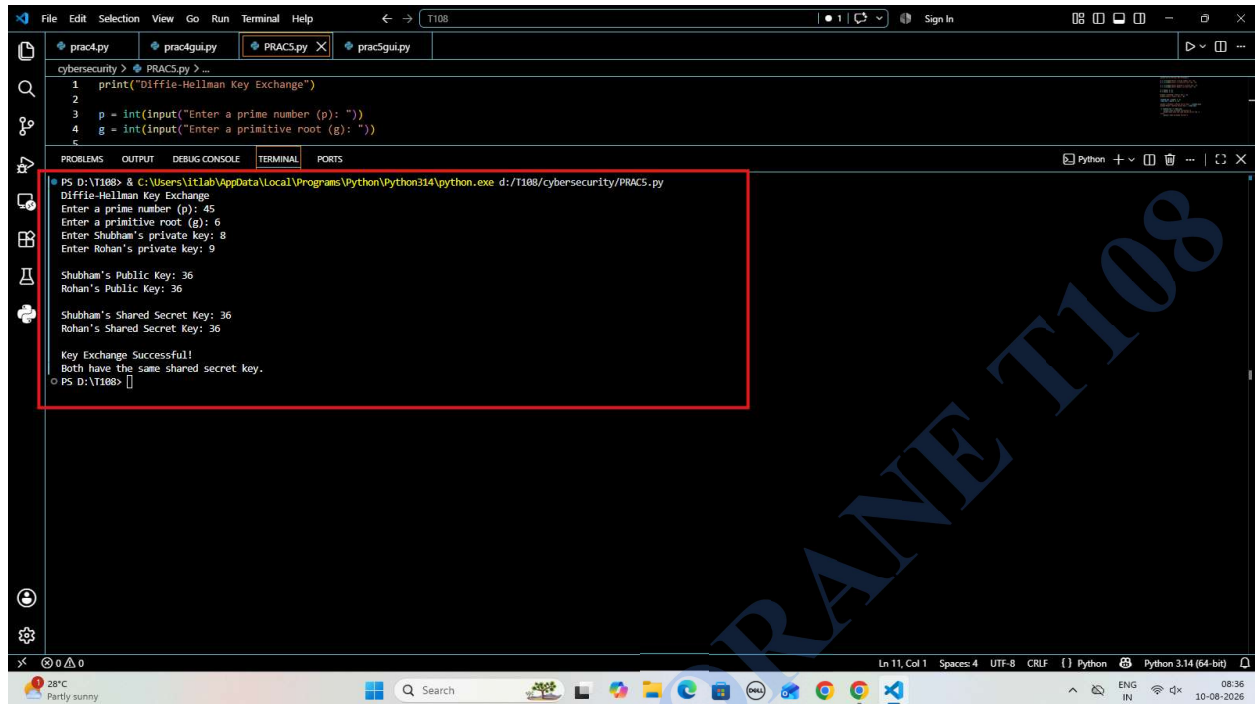
print("\nShubham's Shared Secret Key:", shubham_key)
print("Rohan's Shared Secret Key:", rohan_key)

if shubham_key == rohan_key:
    print("\nKey Exchange Successful!")
    print("Both have the same shared secret key.")
else:
    print("\nKey Exchange Failed!")
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : CYBER SECURITY

OUTPUT



```
cybersecurity> PRACS.py > ...
1 print("Diffie-Hellman Key Exchange")
2
3 p = int(input("Enter a prime number (p): "))
4 g = int(input("Enter a primitive root (g): "))
5
PS D:\T108> & C:\Users\itlab\AppData\Local\Programs\Python\Python314\python.exe d:/T108/cybersecurity/PRACS.py
Diffie-Hellman Key Exchange
Enter a prime number (p): 45
Enter a primitive root (g): 6
Enter Shubham's private key: 8
Enter Rohan's private key: 9

Shubham's Public Key: 36
Rohan's Public Key: 36

Shubham's Shared Secret Key: 36
Rohan's Shared Secret Key: 36

Key Exchange Successful!
Both have the same shared secret key.
PS D:\T108> []
```

CODE GUI:

```
import tkinter as tk
from tkinter import ttk, messagebox
import math

def is_prime(n):
    if n < 2:
        return False

    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False

    return True

def is_primitive_root(g, p):
    if g <= 1 or g >= p:
        return False

    values = set()
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
for i in range(1, p):
    values.add(pow(g, i, p))

return len(values) == p - 1

def generate_keys():

    try:
        p = int(p_entry.get())
        g = int(g_entry.get())
        superman_private = int(superman_entry.get())
        batman_private = int(batman_entry.get())

        if not is_prime(p):
            messagebox.showerror(
                "Invalid Prime Number",
                f"{p} is NOT a prime number.\n\n"
                "Please enter a valid prime number."
            )
            return

        if not is_primitive_root(g, p):
            messagebox.showerror(
                "Invalid Primitive Root",
                f"{g} is NOT a valid primitive root for {p}.\n\n"
                f"Please enter a valid primitive root between 2 and {p -
1}."
            )
            return

        if superman_private <= 0 or superman_private >= p:
            messagebox.showerror(
                "Invalid Superman Key",
                f"Superman's private key must be between 1 and {p - 1}."
            )
            return

        if batman_private <= 0 or batman_private >= p:
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
messagebox.showerror(  
    "Invalid Batman Key",  
    f"Batman's private key must be between 1 and {p - 1}."  
)  
return  
  
if superman_private == batman_private:  
    messagebox.showerror(  
        "Invalid Private Keys",  
        "Superman's and Batman's private keys cannot be the  
same.\n\n"  
        "Please enter two different private keys."  
    )  
    return  
  
superman_public = pow(g, superman_private, p)  
batman_public = pow(g, batman_private, p)  
  
superman_shared = pow(  
    batman_public,  
    superman_private,  
    p  
)  
  
batman_shared = pow(  
    superman_public,  
    batman_private,  
    p  
)  
  
superman_public_result.config(  
    text=f"Superman's Public Key: {superman_public}"  
)  
  
batman_public_result.config(  
    text=f"Batman's Public Key: {batman_public}"  
)  
  
superman_shared_result.config(  
    text=f"Superman's Shared Key: {superman_shared}"
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
)

batman_shared_result.config(
    text=f"Batman's Shared Key: {batman_shared}"
)

if superman_shared == batman_shared:
    status_label.config(
        text="✓ KEY EXCHANGE SUCCESSFUL!",
        foreground="#008000"
    )
else:
    status_label.config(
        text="X KEY EXCHANGE FAILED!",
        foreground="#d00000"
    )

except ValueError:
    messagebox.showerror(
        "Invalid Input",
        "Please enter numbers only."
    )

def clear_fields():

    p_entry.delete(0, tk.END)
    g_entry.delete(0, tk.END)
    superman_entry.delete(0, tk.END)
    batman_entry.delete(0, tk.END)

    superman_public_result.config(
        text="Superman's Public Key: -"
    )

    batman_public_result.config(
        text="Batman's Public Key: -"
    )

    superman_shared_result.config(
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
        text="Superman's Shared Key: -"
    )

    batman_shared_result.config(
        text="Batman's Shared Key: -"
    )

    status_label.config(
        text="Enter values and generate keys",
        foreground="#555555"
    )

window = tk.Tk()

window.title("Superhero Diffie-Hellman Key Exchange")
window.geometry("650x700")
window.configure(bg="#eef5f9")
window.minsize(600, 650)

style = ttk.Style()

try:
    style.theme_use("clam")
except:
    pass

style.configure(
    "Title.TLabel",
    background="#1f4e78",
    foreground="white",
    font=("Arial", 22, "bold")
)

style.configure(
    "Subtitle.TLabel",
    background="#1f4e78",
    foreground="#dceeff",
    font=("Arial", 11)
)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
style.configure(
    "Label.TLabel",
    background="#eef5f9",
    foreground="#222222",
    font=("Arial", 11)
)

style.configure(
    "Entry.TEntry",
    font=("Arial", 12)
)

header = tk.Frame(
    window,
    bg="#1f4e78",
    height=110
)

header.pack(fill="x")

title_label = ttk.Label(
    header,
    text="Diffie-Hellman",
    style="Title.TLabel"
)

title_label.pack(pady=(20, 5))

subtitle_label = ttk.Label(
    header,
    text="Secure Shared Key Generation",
    style="Subtitle.TLabel"
)

subtitle_label.pack()

input_frame = tk.LabelFrame(
    window,
    text="  Input Parameters  ",
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
        bg="#eef5f9",
        fg="#1f4e78",
        font=("Arial", 13, "bold"),
        padx=25,
        pady=20
    )

input_frame.pack(
    padx=50,
    pady=25,
    fill="x"
)

ttk.Label(
    input_frame,
    text="Prime Number (p):",
    style="Label.TLabel"
).grid(
    row=0,
    column=0,
    sticky="w",
    pady=10
)

p_entry = ttk.Entry(
    input_frame,
    width=30,
    style="Entry.TEntry"
)

p_entry.grid(
    row=0,
    column=1,
    padx=20,
    pady=10
)

ttk.Label(
    input_frame,
    text="Primitive Root (g):",
```


SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
        style="Label.TLabel"
    ).grid(
        row=1,
        column=0,
        sticky="w",
        pady=10
    )

    g_entry = ttk.Entry(
        input_frame,
        width=30,
        style="Entry.TEntry"
    )

    g_entry.grid(
        row=1,
        column=1,
        padx=20,
        pady=10
    )

    ttk.Label(
        input_frame,
        text="Superman's Private Key:",
        style="Label.TLabel"
    ).grid(
        row=2,
        column=0,
        sticky="w",
        pady=10
    )

    superman_entry = ttk.Entry(
        input_frame,
        width=30,
        style="Entry.TEntry"
    )

    superman_entry.grid(
        row=2,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
        column=1,  
        padx=20,  
        pady=10  
    )  
  
    ttk.Label(  
        input_frame,  
        text="Batman's Private Key:",  
        style="Label.TLabel"  
    ).grid(  
        row=3,  
        column=0,  
        sticky="w",  
        pady=10  
    )  
  
    batman_entry = ttk.Entry(  
        input_frame,  
        width=30,  
        style="Entry.TEntry"  
    )  
  
    batman_entry.grid(  
        row=3,  
        column=1,  
        padx=20,  
        pady=10  
    )  
  
    button_frame = tk.Frame(  
        window,  
        bg="#eef5f9"  
    )  
  
    button_frame.pack(pady=5)  
  
    generate_button = tk.Button(  
        button_frame,  
        text="🔑 Generate Keys",  
        command=generate_keys,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
        bg="#16803c",
        fg="white",
        font=("Arial", 12, "bold"),
        padx=25,
        pady=10,
        relief="flat",
        cursor="hand2"
    )

generate_button.grid(
    row=0,
    column=0,
    padx=10
)

clear_button = tk.Button(
    button_frame,
    text="🗑 Clear",
    command=clear_fields,
    bg="#d9534f",
    fg="white",
    font=("Arial", 12, "bold"),
    padx=25,
    pady=10,
    relief="flat",
    cursor="hand2"
)

clear_button.grid(
    row=0,
    column=1,
    padx=10
)

output_frame = tk.LabelFrame(
    window,
    text=" Results ",
    bg="white",
    fg="#1f4e78",
    font=("Arial", 13, "bold"),
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
        padx=25,  
        pady=20  
    )  
  
    output_frame.pack(  
        padx=50,  
        pady=25,  
        fill="x"  
    )  
  
    superman_public_result = tk.Label(  
        output_frame,  
        text="Superman's Public Key: -",  
        bg="white",  
        font=("Arial", 12)  
    )  
  
    superman_public_result.pack(pady=7)  
  
    batman_public_result = tk.Label(  
        output_frame,  
        text="Batman's Public Key: -",  
        bg="white",  
        font=("Arial", 12)  
    )  
  
    batman_public_result.pack(pady=7)  
  
    superman_shared_result = tk.Label(  
        output_frame,  
        text="Superman's Shared Key: -",  
        bg="white",  
        font=("Arial", 12)  
    )  
  
    superman_shared_result.pack(pady=7)  
  
    batman_shared_result = tk.Label(  
        output_frame,  
        text="Batman's Shared Key: -",
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

```
bg="white",
font=("Arial", 12)
)

batman_shared_result.pack(pady=7)

status_label = tk.Label(
    window,
    text="Enter values and generate keys",
    bg="#eef5f9",
    fg="#555555",
    font=("Arial", 14, "bold")
)

status_label.pack(pady=10)

window.mainloop()
```

OUTPUT

Superhero Diffie-Hellman Key Exchange

Diffie-Hellman

Secure Shared Key Generation

Input Parameters

Prime Number (p):

Primitive Root (g):

Superman's Private Key:

Batman's Private Key:

Results

Superman's Public Key: 4

Batman's Public Key: 28

Superman's Shared Key: 39

Batman's Shared Key: 39

✓ KEY EXCHANGE SUCCESSFUL!

28°C Partly sunny | Search | 10-08-2026 08:43

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : CYBER SECURITY

The image displays two screenshots of a web application titled "Diffie-Hellman Secure Shared Key Generation".

Top Screenshot:

- Input Parameters:**
 - Prime Number (p): 88
 - Primitive Root (g): 7
 - Superman's Private Key: 8
 - Batman's Private Key: 9
- Results:**
 - Superman's Public Key: 4
 - Batman's Public Key: 28
 - Superman's Shared Key: 39
 - Batman's Shared Key: 39
- Status:** KEY EXCHANGE SUCCESSFUL!

An error dialog box titled "Invalid Prime Number" is displayed, stating: "88 is NOT a prime number. Please enter a valid prime number." with an "OK" button.

Bottom Screenshot:

- Input Parameters:**
 - Prime Number (p): 89
 - Primitive Root (g): 7
 - Superman's Private Key: 8
 - Batman's Private Key: 8
- Results:**
 - Superman's Public Key: 4
 - Batman's Public Key: 28
 - Superman's Shared Key: 39
 - Batman's Shared Key: 39
- Status:** KEY EXCHANGE SUCCESSFUL!

An error dialog box titled "Invalid Private Keys" is displayed, stating: "Superman's and Batman's private keys cannot be the same. Please enter two different private keys." with an "OK" button.

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

Description

The Diffie-Hellman Key Exchange algorithm is a cryptographic method used by two users to create a common secret key over an insecure network. In this practical, Superman and Batman represent the two users.

Both users agree on a prime number p and a primitive root g . Each user selects a private key and calculates a public key. The public keys are exchanged, and each user uses the other user's public key along with their own private key to calculate the shared secret key.

Although the private keys are never shared, both users generate the same shared secret key.

The GUI also checks whether the entered values are valid. It displays an error if the prime number is invalid, the primitive root is invalid, or both users enter the same private key.

Algorithm

1. Start the program.
2. Enter a prime number p .
3. Enter a valid primitive root g .
4. Enter Superman's private key.
5. Enter Batman's private key.
6. Check whether p is prime.
7. Check whether g is a valid primitive root.
8. Check that both private keys are valid and different.
9. Calculate Superman's public key:
$$A = g^a \text{ mod } p$$
10. Calculate Batman's public key:
$$B = g^b \text{ mod } p$$
11. Calculate Superman's shared key:
$$K1 = B^a \text{ mod } p$$
12. Calculate Batman's shared key:
$$K2 = A^b \text{ mod } p$$
13. Compare $K1$ and $K2$.
14. If both keys are equal, display "Key Exchange Successful".
15. Otherwise, display "Key Exchange Failed".

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : CYBER SECURITY

16. Stop.

Short Flow

Enter p and g



Enter Superman's private key



Enter Batman's private key



Validate inputs



Generate public keys



Exchange public keys



Generate shared keys



Compare shared keys



SUCCESS / FAILED